Introduction to IC Verification

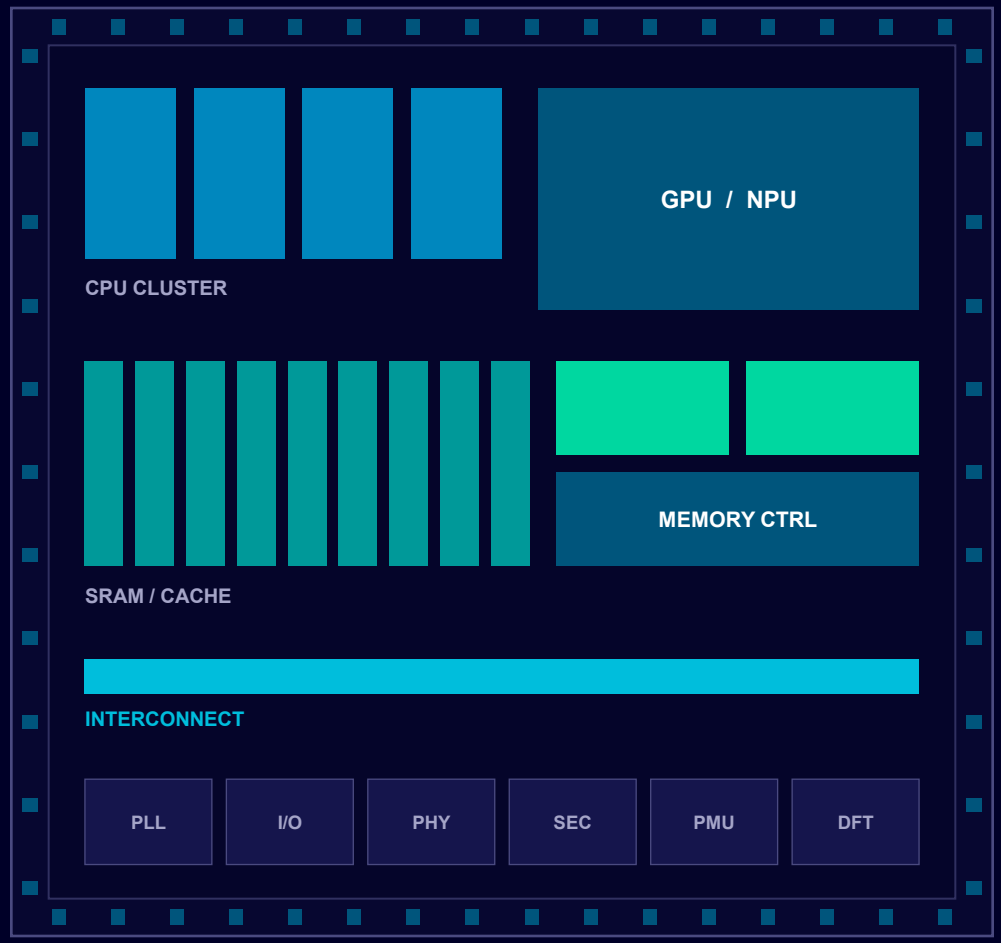
Amit Tanwar
Binoy Oommen

August 5, 2026

SIEMENS

The Invisible Engineering

Why every chip you will ever use had to be proven correct before it existed.



A modern SoC floorplan. Every block on it was proven in software before a single atom was deposited.

SIEMENS

Two hours, six questions

No syntax today. Today is the map - the shape of the industry you are potentially walking into.

01

Where does verification sit in the IC design cycle?

02

How is verification actually different from design?

03

Why does verification decide silicon success?

04

What does the verification life cycle look like?

05

Simulation or formal verification?

06

Simulation or emulation?

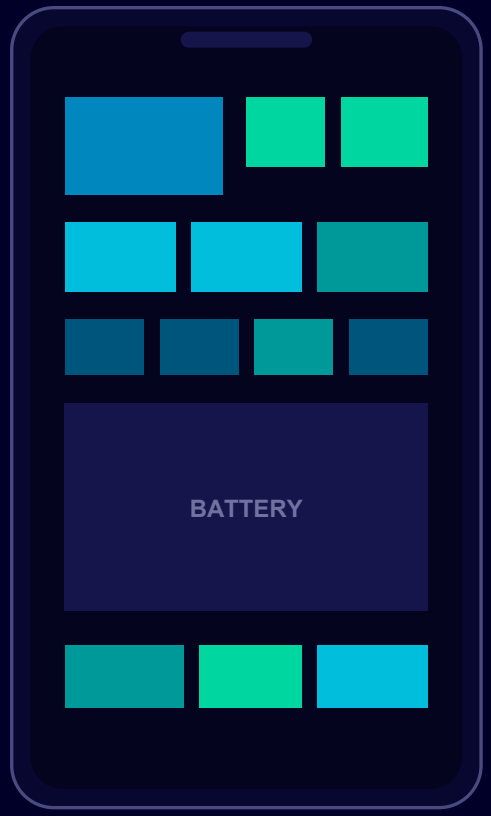
SIEMENS

The world you are walking into

Before we talk about verification, look at what is actually being built - and at the scale of the bet being placed on it.

Empty your pocket

One product. Fifteen to twenty separate silicon devices. Fifteen to twenty separate verification efforts.



- Application processor (SoC)
- 5G modem
- RF transceiver + front-end
- Power management IC
- DRAM
- NAND flash storage
- Image signal processor
- Camera image sensors (x3-4)
- Display driver IC
- Touch controller
- Audio codec + amplifier
- Wi-Fi / Bluetooth combo
- NFC controller
- Secure element
- Sensor hub / IMU
- Battery charger / fuel gauge

Every one of them shipped with a bug list that somebody decided was acceptable. Verification is how that decision gets made.

SIEMENS

ONE CHIP

208,000,000,000

transistors on a single NVIDIA Blackwell B200 GPU

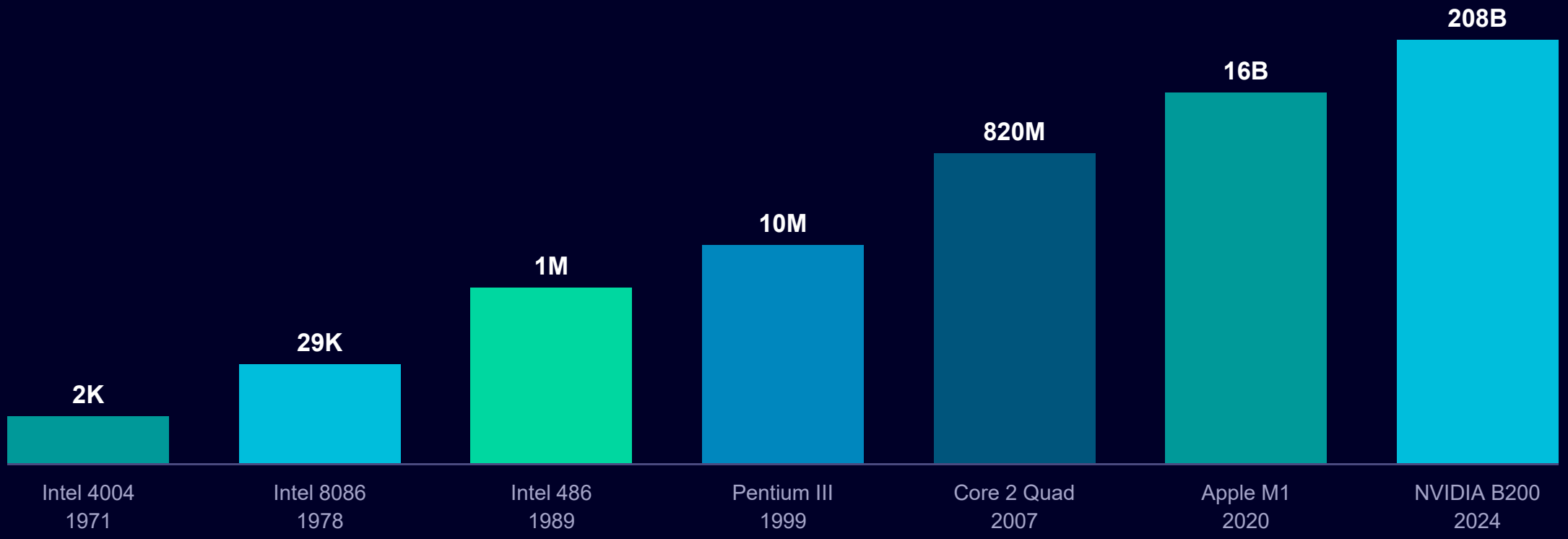
Count them at one per second, without stopping, and you finish in about 6,600 years.

No human reviews this. No human can. Confidence at this scale has to be manufactured by machines, under a plan, with a number at the end of it.

Transistor count: NVIDIA, publicly reported, 2024.

Fifty years of doubling

Logarithmic scale. Every step up the axis is ten times more transistors.

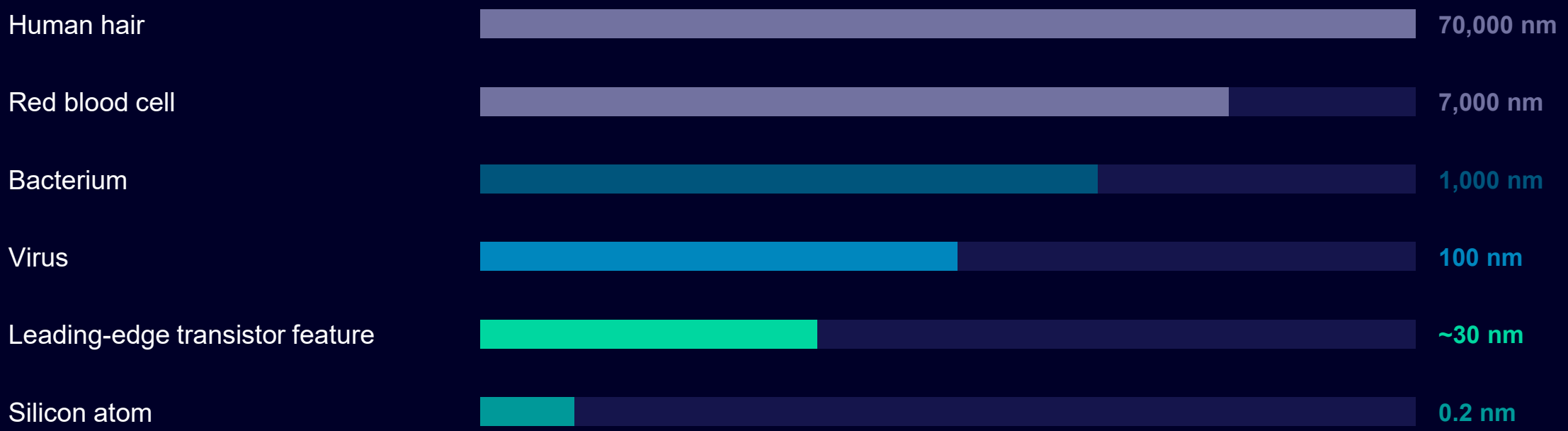


The design got roughly 90 million times more complex. The tolerance for a wrong answer did not move.

SIEMENS

And it all happens down here

Logarithmic scale, nanometres.



You cannot probe it, cut it, patch it or single-step it. Once it is fabricated, almost everything inside is permanently invisible - so all the looking has to happen beforehand, in software.

SIEMENS

From sand to your hand

Roughly 12 to 16 weeks in the fab, across hundreds of process steps. That is your feedback loop.

If the logic is wrong, you find out one financial quarter later - and the mask set you already paid for is scrap.

SIEMENS

This is not a niche, and it is not far away

And for the first time, a serious part of it is being built in this country - with a talent plan you are already inside.

$795.6 B

Global semiconductor sales, 2025

Up 26% year on year (WSTS). Logic alone was $302 B.

Rs 1.64 lakh crore

Approved under the India Semiconductor Mission

12 projects: fabs, compound-semi fabs and packaging/test units.

85,000

Semiconductor engineers India is training

EDA tools live in 315 institutions today, expanding to 500.

You are not reading about that plan. You are inside it.

Sources: WSTS annual figures for 2025; India Semiconductor Mission / MeitY Chips-to-Startup programme announcements, 2026.

SIEMENS

Nobody draws a chip

You describe the behaviour you want in a language. Software turns that into two hundred billion transistors.

ALL OF IT RUNS ON EDA SOFTWARE

Electronic Design Automation

A software industry of a few tens of billions of dollars that gates a semiconductor industry of nearly eight hundred. Three companies dominate it. Siemens EDA is one of them.

The verification corner of it

Questa
simulation

Questa Formal
formal proof

Veloce
emulation

Questa VIP
protocol verification IP

Questa Student Edition will be installed on the DTU lab machines - you start driving it next week.

SIEMENS

No chip has ever been built that was not first run in software.

So the real question is never "does it work?" It is "how much of it did we actually try - and how would we know if we missed something?"

SIEMENS

Where does the effort actually go?

Siemens EDA / Wilson Research Group functional verification study - the industry's long-running census.

SHARE OF TOTAL IC / ASIC PROJECT TIME

Essentially unchanged since 2008.

~1 : 1

Peak verification engineers to design engineers, averaged across market segments.

up to 5 : 1

On processor projects, verification headcount routinely outnumbers design five to one.

~49%

Of a design engineer's own time is spent on verification activity, not design.

If you thought verification was the consolation prize, the industry disagrees with you.

SIEMENS

14%

of IC / ASIC projects achieve first silicon success.

The lowest level recorded in two decades.

Read that again.

Seventy per cent of the effort. Some of the best engineers alive. Decades of tooling.

And roughly six chips out of seven still need another mask set.

That is not a failure of verification. That is the size of the problem you are being trained to attack.

2024 Siemens EDA / Wilson Research Group IC-ASIC functional verification trend report.

PART 01

Where verification lives

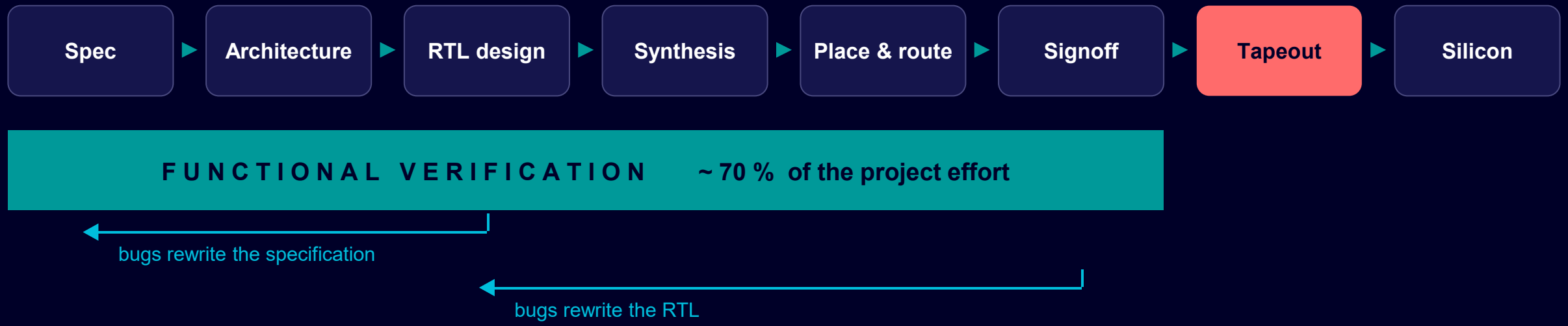
01

Role of verification in the IC design cycle - and why "tapeout" is the most frightening word in this industry.

SIEMENS

The IC design cycle

Verification is not a box on this line. It is the band underneath it.



It starts at the specification.

The verification plan is written from the spec, in parallel with the design - ideally by someone who is not the designer.

It runs backwards as well as forwards.

Most bugs found are not "the RTL is wrong". They are "the specification never actually said".

It ends in a judgement, not a tick.

Somebody signs their name against "the remaining risk is acceptable". That signature is the deliverable.

SIEMENS

Tapeout is a one-way door

The moment the design database is released to the foundry. From here, the logic is frozen in glass and metal.

SAME BUG. DIFFERENT DAY.

Found before tapeout

~2 days · edit the RTL, rerun the regression

Found after tapeout

3 - 6 months · new masks, new fab run, new bring-up

Both bars drawn to the same scale.

The mask set

A leading-edge mask set runs to millions of dollars. A respin throws the old one away.

The fab queue

12-16 weeks of processing before anybody powers it up - every single time.

The market window

Miss the design-in window for a phone or a car platform and the product may simply not have a customer any more.

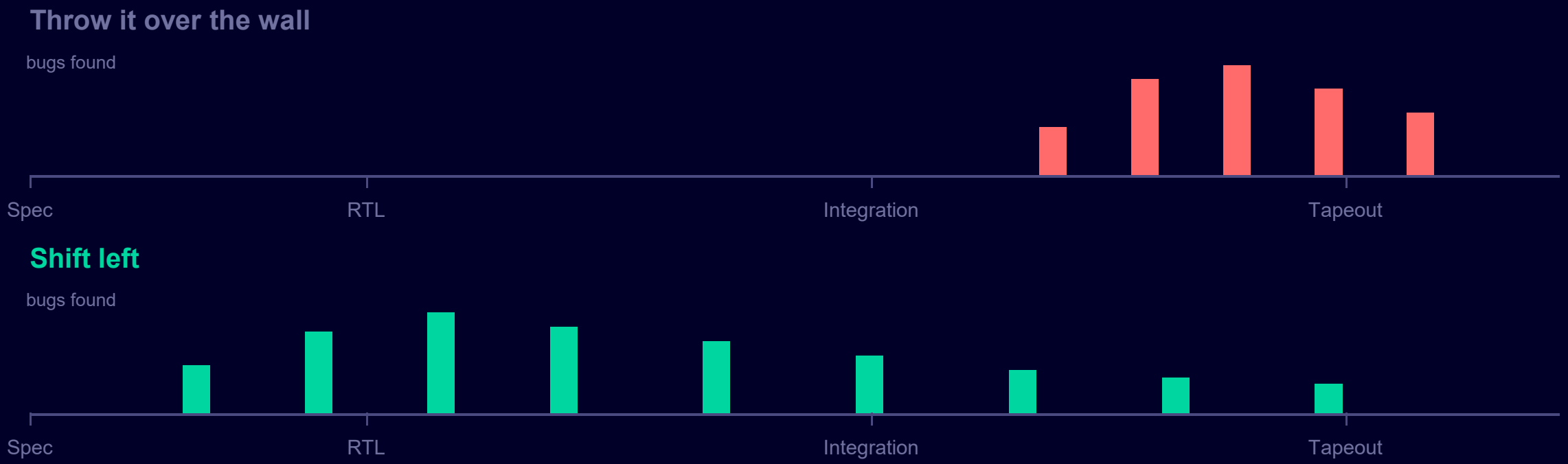
Every level of design has a matching level of proof

You cannot find a FIFO overflow by booting an operating system, and you cannot prove a system requirement by testing one module. Match the level to the question.

SIEMENS

Shift left

The cheapest bug in the world is the one you find in the specification review.



> The real prize is not earlier bug reports. It is better specifications - because somebody whose entire job is asking "what happens if..." reads the document while it can still be changed.

SIEMENS

PART 02

Two jobs, one specification

02

Verification versus design. Not "builder and tester" - two independent readings of the same sentence.

SIEMENS

Design and verification

DESIGN

THE QUESTION

How do I build it?

WHAT YOU WRITE

RTL - synthesisable hardware behaviour

OPTIMISES FOR

Power, performance, area, timing closure

"DONE" MEANS

It implements the spec and meets timing

MINDSET

Make it work

VERIFICATION

THE QUESTION

How do I know it is right?

WHAT YOU WRITE

A testbench - stimulus, checkers, coverage

OPTIMISES FOR

Confidence, completeness, reproducibility

"DONE" MEANS

The evidence says the required behaviour was exercised and checked

MINDSET

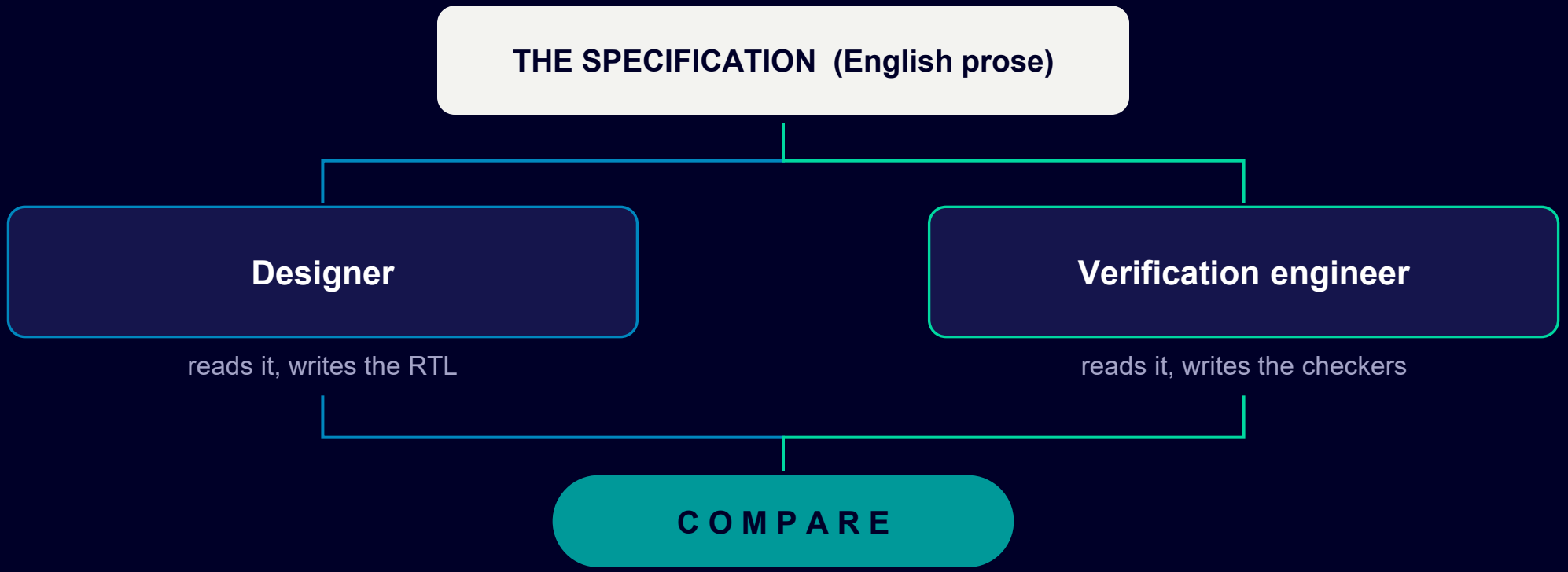
Make it fail

A verification engineer who runs a clean regression and feels good about it has misunderstood the job.

SIEMENS

Why it has to be two people

Verification does not work because we are cleverer. It works because of redundancy.



They agree
A little confidence. Only a little.

They disagree
A bug in the RTL, the testbench, or the spec.

SIEMENS

One sentence. Six unanswered questions.

"The FIFO shall assert full when it cannot accept further data."

- Is full asserted in the same cycle as the last write, or the next one?
- What if a read and a write arrive in the same cycle while full?
- Does full deassert combinatorially on a read, or one cycle later?
- What is the value of full during and immediately after reset?
- Is a write while full silently ignored, or does it corrupt data?
- What does any of this mean if the depth is configured to 1?

The deepest bugs are never "the code is wrong". They are "the spec never said, and two people assumed differently."

PAIR EXERCISE · 7 MINUTES

Break this specification

> ***"On receiving three consecutive invalid PIN entries, the lock shall remain closed for 60 seconds."***

1

In pairs, 4 minutes

Find four questions this sentence does not answer.

2

Then, 1 minute

Pick the one you would bet is a bug in a shipping product.

3

Debrief, 2 minutes

We put them on the board. No code required to find a bug.

Hint: think about time, about reset, about power loss - and about somebody who is deliberately attacking the lock.

SIEMENS

BREAK

10 minutes

When we come back:

What does it actually cost when this goes wrong?

SIEMENS

PART 03

The cost of being wrong

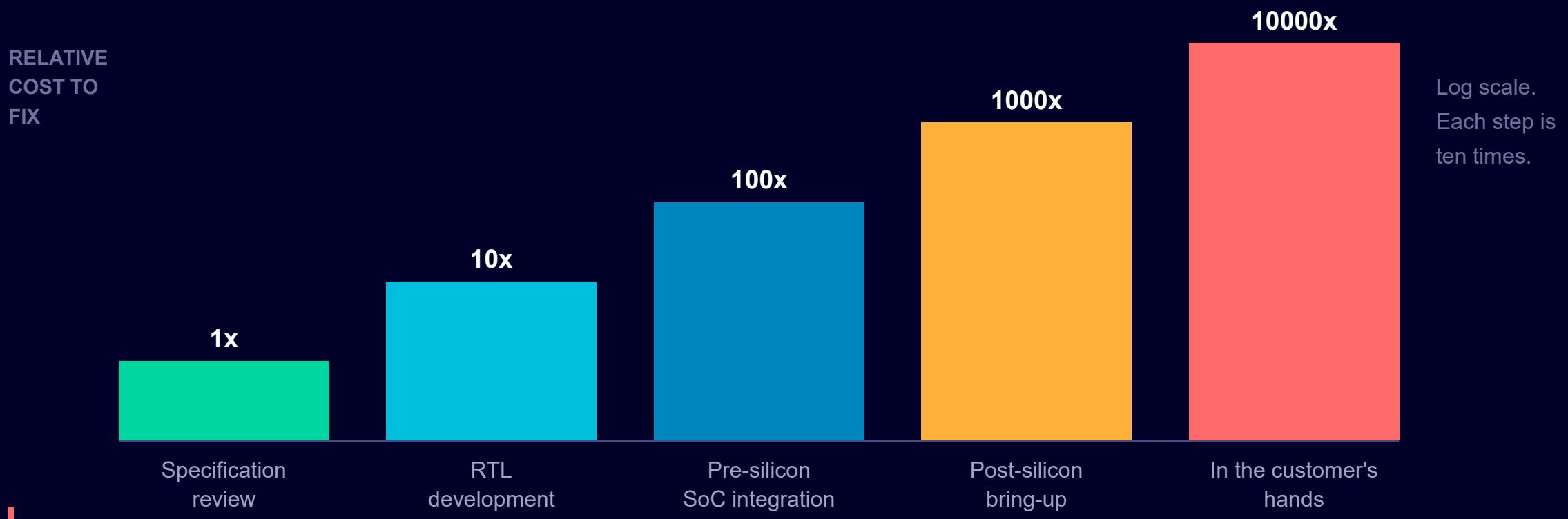
03

Importance of verification in silicon success - told through three bugs that made the news.

SIEMENS

The same bug, priced by the day you find it

Illustrative orders of magnitude, not a price list. The shape is the point.



Roughly ten thousand to one, first bar to last. The bug did not change. Only the day you found it changed. That arithmetic is the whole reason verification gets 70% of the budget.

SIEMENS

Intel Pentium, 1994

The bug that made verification a boardroom subject.

```
4195835   /   3145727   =
```

1.333820449 correct

1.333739068 what the Pentium returned

No crash. No hang. A plausible wrong answer, returned silently.

THE CAUSE

Entries missing from the lookup table used by the floating-point divider.

THE ODDS

Roughly 1 in 9 billion randomly chosen operand pairs, by Intel's own estimate.

THE COST

$475 million pre-tax charge, and Intel's first ever CPU recall.

A directed test walking the entries of that table would have found it in an afternoon. Nobody wrote it.

So the real question is not "why did they miss it?" but "how would you have known to write that test?" - and the answer is coverage, which is Unit 4 of your syllabus.

Intel 1994 annual report; Pentium FDIV bug, widely documented.

It is not always the logic

Intel Cougar Point

2011

One transistor in the SATA circuit leaked slightly too much current. The ports degraded over months. It was perfect on day one.

$700 M to repair and replace, plus ~$300 M of halted revenue.

How do you test for something that only fails after two years?

Spectre / Meltdown

2018

The RTL was correct. It implemented the architecture exactly. Speculative execution simply left traces in cache timing.

A whole industry patched and paid for it in performance.

You can only verify against what somebody thought to specify.

The common thread

Different eras, different companies, different root causes - logic, circuit behaviour, and specification blindness.

None of them were sloppy teams. All of them were world class.

Every one of these passed its own tests.

What "silicon success" actually means

The device meets its functional, performance, power and reliability specification - in every legal configuration, in the customer's real system, on the first mask set.

"every legal configuration"

A configurable IP with twelve parameters has thousands of legal builds. You will never simulate them all.

"in the customer's real system"

Their firmware, their board, their temperature, their corner cases - none of which you had on your desk.

"on the first mask set"

One attempt. There is no staging environment and no hotfix branch.

And no significant chip ever ships bug-free.

Every complex device ships with a published errata list - known bugs the team chose to live with. The deliverable of verification is not "no bugs". It is: we know what is in there, we chose knowingly, and nothing unknown is waiting for us.

PART 04

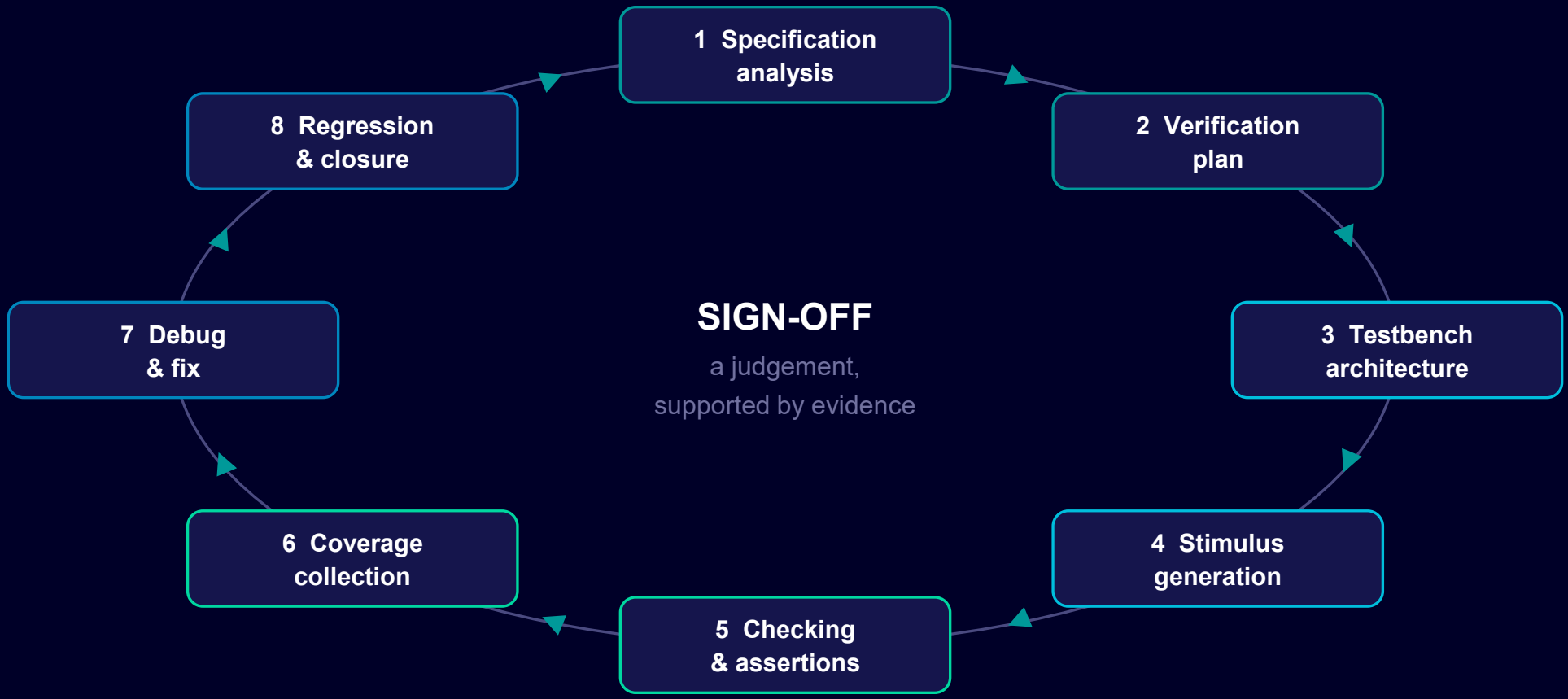
How the work is actually done

04

The verification life cycle - and the six artefacts you will personally build over the next fifteen weeks.

SIEMENS

The verification life cycle



CHECKING asks "was it correct?" · COVERAGE asks "did we ever try it?" · You need both.

SIEMENS

Six artefacts. You will build all of them.

Every stage of that loop produces something concrete - and every one of them is a unit of this syllabus.

Verification plan

Unit 1

Every requirement mapped to how it will be stimulated and how it will be checked.

Protocol drivers & monitors

Unit 2

The components that speak APB, AHB-Lite, AXI4-Lite, SPI, I2C to your design.

Layered testbench

Units 3 & 5

Classes, interfaces, packages - the reusable structure that makes all of it scale.

Constrained-random stimulus

Unit 4

Randomisation that stays legal, so the machine explores cases you never thought of.

Functional coverage model

Unit 4

The measurement that tells you what you actually exercised - and what you never did.

Assertions (SVA)

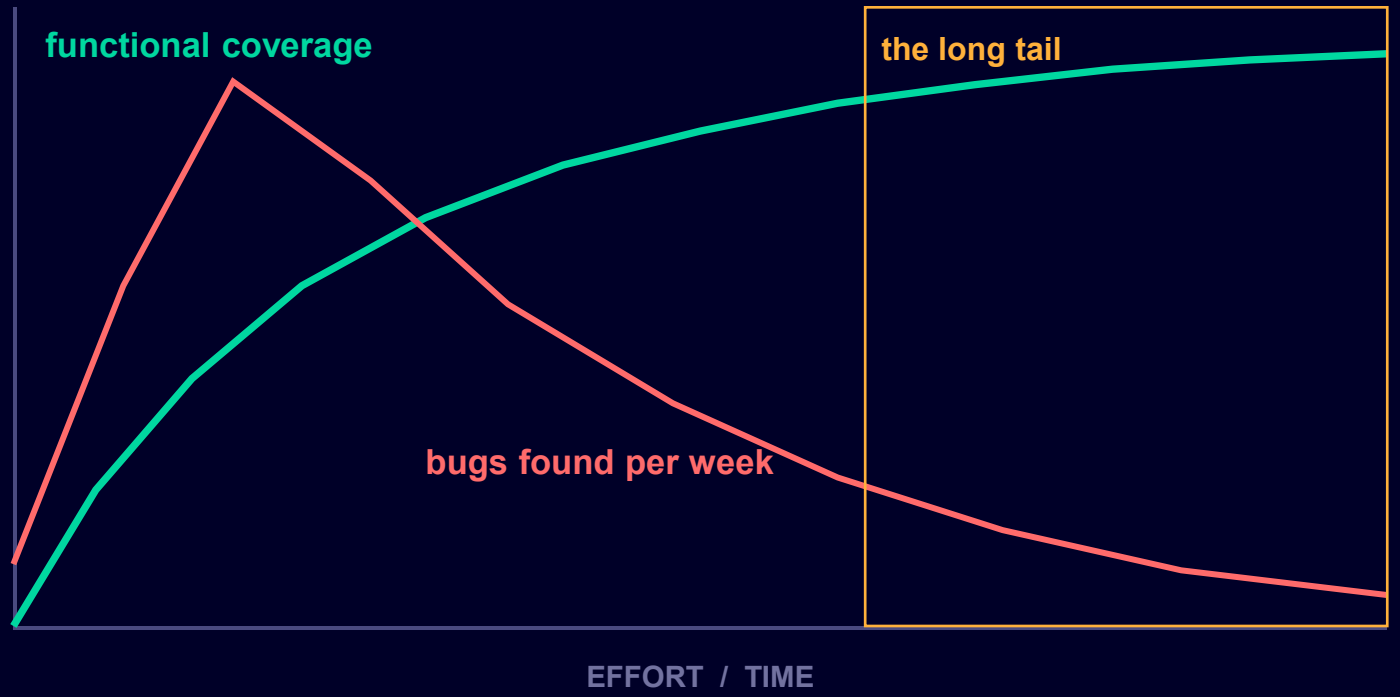
Unit 5

Temporal rules that watch the design continuously and fail the instant they are violated.

By the industry code contest in week 15, you will have built every one of these for a real problem statement.

SIEMENS

So when are you actually done?



The trap

A falling bug rate is ambiguous. It might mean the design is clean - or it might mean your stimulus stopped exploring anything new.

The only way to tell those two apart is coverage. That is what coverage is actually for.

"Done" = coverage goals met, all checks passing, regression stable, and a documented list of what was deliberately not covered. A risk judgement, with evidence, signed by a person.

SIEMENS

PART 05

Two ways to hunt a bug

05

Simulation runs the scenarios you thought of. **Formal** proves the ones you did not.

SIEMENS

Simulation and formal, in one picture

SIMULATION / DYNAMIC

You write STIMULUS. Each line is one test walking one path. Run millions of them and look at how much space is left. It can show bugs are present - never that they are absent.

FORMAL

You write a PROPERTY. The tool searches every reachable state to a bounded depth, under your assumptions. It proves the claim - or hands you the shortest trace that breaks it.

A design with 100 flip-flops has more reachable states than there are atoms in the observable universe. Neither method is optional.

SIEMENS

Choosing between them

	SIMULATION	FORMAL
What you write	Stimulus, checkers and coverage	Properties and assumptions
What it answers	"Did I find a bug?"	"Can a bug exist at all?"
What you get out	A pass, or a failing trace to debug	A proof, or the shortest counterexample
Strongest on	Realistic traffic, datapaths, whole-system behaviour	Control logic, arbiters, protocol FSMs, "never" and security rules
Blind spot	The case you did not think to stimulate	Depth limits, and assumptions you got wrong
In this course	Units 1-5, and the code contest	Unit 5, with SVA

Nobody picks one. Formal on the control-heavy blocks and the security rules, simulation everywhere else.

A formal property, in four lines

You will meet this properly in Unit 5. For now, just notice how close to English it is.

```
property no_grant_during_lockout;
  @(posedge clk) disable iff (!rst_n)
    lockout |-> !grant;
endproperty

assert property (no_grant_during_lockout);
```

"At every rising clock edge, unless we are in reset: if lockout is asserted, then grant must not be."

SIMULATION SAYS

"I ran ten million random PIN sequences and grant never asserted during lockout."

FORMAL SAYS

"grant CANNOT assert during lockout. Here is the proof - or here is a seven-cycle trace where it does."

Those are not the same sentence. One is evidence. The other is proof.

This is the smart lock from your pair exercise - the same ambiguity, now pinned down into something a machine can check exhaustively.

PART 06

Same design, different engine

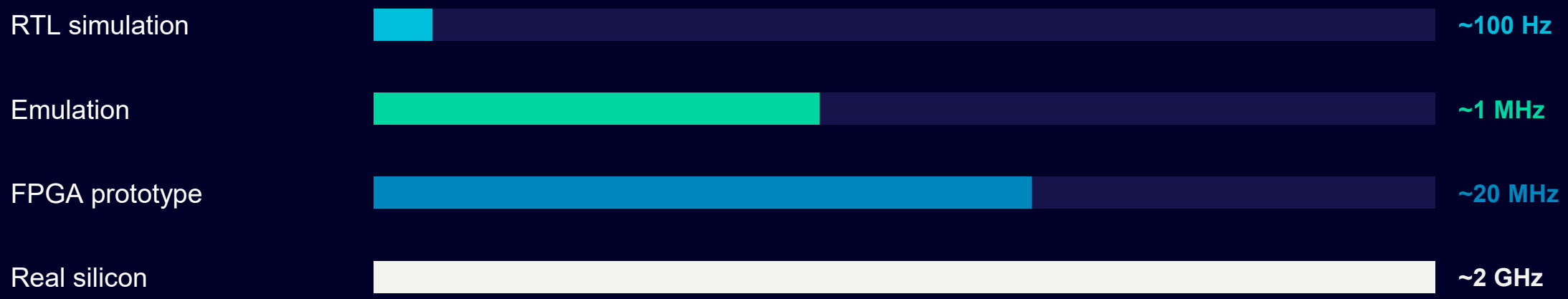
06

Same RTL, same tests, same checkers. The only difference is what is executing them - and that is five orders of magnitude.

SIEMENS

Simulation or emulation? It is a speed problem.

Same RTL. Same design. The difference is what is executing it.



Effective clock rate for a large SoC. Log scale.

BOOTING AN OPERATING SYSTEM ≈ 10 BILLION CLOCK CYCLES

RTL simulation	Emulation	FPGA prototype	Real silicon
~3 years	~3 hours	~8 minutes	5 seconds

Illustrative order-of-magnitude arithmetic. You cannot boot an operating system in simulation - not slowly, at all.

SIEMENS

The trade you are making

SIMULATION

Software executing your RTL on a CPU

- Every internal signal visible, any time
- Edit, recompile and rerun in minutes
- Cheap - it runs on the machines you already have
- Where you will live day to day
- Far too slow to run real software workloads

EMULATION

Your RTL compiled onto purpose-built hardware

- Four to five orders of magnitude faster
- Boots real firmware and real operating systems
- Hardware and software debugged together, before silicon
- Substantial capital cost and long compile times
- Much more constrained visibility when it fails

You switch to emulation when the bug you are hunting needs a billion cycles of real software to reproduce. Not before.

SIEMENS

Six questions, six answers

You answer them first. Then we compare.

01

A band underneath the whole cycle, from specification to tapeout - not a phase at the end.

02

Design implements the spec. Verification independently interprets the same spec and compares.

03

Because tapeout is a one-way door, and the same bug costs ten thousand times more on the wrong side of it.

04

Plan, build, stimulate, check, measure coverage, debug, regress - and go round again.

05

Simulation finds bugs in the cases you thought of. Formal proves they cannot exist, within bounds.

06

Same RTL, different execution engine. Simulation to debug, emulation when you need to run real software.

SIEMENS

Next: the language all of this is written in

YOU ARE HERE

Unit 1
Verification fundamentals and planning

Unit 2
Bus protocols and verification requirements

Unit 3
SystemVerilog and OOP foundations

Unit 4
Constrained random and functional coverage

Unit 5
Assertions, testbench architecture, AI debug

Week 15
Industry verification code contest

Before next week

Pick any device you own. Write down two behaviours its hidden electronics must get right every single time. Bring them - we will turn one into a verification plan entry in four minutes.

SIEMENS

In 1994, a handful of missing table entries cost Intel $475 million.

Somebody could have written that test.

In fifteen weeks, that somebody is you.

Assignment: Verilog testbench of a real-life design, with AI as your co-designer

"My phone screen saver must be activated within its configured timer when it sees no activity."

01 Design it

- Take the real-life behaviour from U1-1 - the one you just wrote down.
- Ask AI to design the Verilog control unit for it.

02 Analyze it

- Analyse what came back: input and output signals, functionality, corner cases.
- Ask AI to modify until it does what you actually meant.

03 Verify it

- Ask AI to generate a verification plan for that design.
- Then judge the plan yourself. Is it complete? What did it miss?

SIEMENS